```python
import threading

def task1():

    for i in range(1, 6):
        print("Thread 1:", i)


def task2():

    for i in range(1, 6):
        print("Thread 2:", i)


t1 = threading.Thread(target=task1)
t2 = threading.Thread(target=task2)

t1.start()
t2.start()

t1.join()
t2.join()

print("Both threads completed")
```

OUTPUT

```
= RESTART: C:/Users/ATHIN/AppDa
sssssssssssssssssssssssssssss
Thread 1:Thread 2:  11

Thread 1:Thread 2:  22

Thread 1:Thread 2:  33

Thread 1:Thread 2:  44

Thread 1:Thread 2:  55

Both threads completed
```

```python
import threading
import time

n = 5

forks = []

for i in range(n):
    forks.append(threading.Lock())


def philosopher(i):

    left = i
    right = (i + 1) % n

    print("Philosopher", i + 1, "is thinking")

    if i == n - 1:
        first = right
        second = left
    else:
        first = left
        second = right

    forks[first].acquire()
    forks[second].acquire()

    print("Philosopher", i + 1, "is eating")

    time.sleep(1)

    forks[second].release()
    forks[first].release()

    print("Philosopher", i + 1, "finished eating")


threads = []
```

OUTPUT

```
= RESTART: C:/Users/ATHIN/AppData/Local/Programs/Python/Python313/a
sssssssssssssssssssssssssssssssssssssssssssssssssssss.py
PhilosopherPhilosopherPhilosopherPhilosopherPhilosopher     13245
  is thinkingis thinkingis thinkingis thinkingis thinking




PhilosopherPhilosopher  13  is eatingis eating

PhilosopherPhilosopherPhilosopherPhilosopher    5123    is eatingfi
nished eatingis eatingfinished eating



PhilosopherPhilosopher  Philosopher54   2finished eatingis eating

finished eating
Philosopher 4 finished eating
All philosophers finished
```

```
def first_fit(blocks, processes):

    b = blocks.copy()
    allocation = [-1] * len(processes)

    for i in range(len(processes)):

        for j in range(len(b)):

            if b[j] >= processes[i]:

                allocation[i] = j + 1
                b[j] -= processes[i]

                break

    return allocation


def best_fit(blocks, processes):

    b = blocks.copy()
    allocation = [-1] * len(processes)

    for i in range(len(processes)):

        best = -1

        for j in range(len(b)):

            if b[j] >= processes[i]:

                if best == -1 or b[j] < b[best]:
                    best = j

        if best != -1:

            allocation[i] = best + 1
            b[best] -= processes[i]
```

OUTPUT

```
= RESTART: C:/Users/ATHIN/AppData/Local/Pro
sssssssssssssssssssssssssssssssssssssssssss
Enter block sizes separated by space: 3
Enter process sizes separated by space: 4

First Fit : [-1]
Best Fit  : [-1]
Worst Fit : [-1]
```

```
files = []

while True:

    print("\n1. Create File")
    print("2. Delete File")
    print("3. Display Files")
    print("4. Exit")

    choice = int(input("Enter choice: "))

    if choice == 1:

        name = input("Enter file name: ")

        if name in files:
            print("File already exists")
        else:
            files.append(name)
            print("File created successfully")

    elif choice == 2:

        name = input("Enter file name: ")

        if name in files:
            files.remove(name)
            print("File deleted")
        else:
            print("File not found")

    elif choice == 3:

        print("\nFiles in Directory:")

        if len(files) == 0:
            print("Directory is empty")
        else:
            for file in files:
                print(file)
```

OUTPUT

```
1. Create File
2. Delete File
3. Display Files
4. Exit
Enter choice: 1
Enter file name: ATHIN
File created successfully

1. Create File
2. Delete File
3. Display Files
4. Exit
Enter choice: 1
Enter file name: ATHI
File created successfully

1. Create File
2. Delete File
3. Display Files
4. Exit
Enter choice: 3

Files in Directory:
ATHIN
ATHI

1. Create File
2. Delete File
3. Display Files
```

```
directories = {}

while True:

    print("\n1. Create User Directory")
    print("2. Create File")
    print("3. Display Directories")
    print("4. Exit")

    choice = int(input("Enter choice: "))

    if choice == 1:

        user = input("Enter user name: ")

        if user in directories:
            print("Directory already exists")
        else:
            directories[user] = []
            print("Directory created")

    elif choice == 2:

        user = input("Enter user name: ")

        if user not in directories:
            print("Directory not found")

        else:

            filename = input("Enter file name: ")

            if filename in directories[user]:
                print("File already exists")

            else:
                directories[user].append(filename)
                print("File created")

    elif choice == 3:
```

OUTPUT

```
SSSSSSSSSSSSSSSSSSSSSSSSSSSSS

1. Create User Directory
2. Create File
3. Display Directories
4. Exit
Enter choice: 1
Enter user name: ATHIN
Directory created

1. Create User Directory
2. Create File
3. Display Directories
4. Exit
Enter choice: 2
Enter user name: ARUN
Directory not found

1. Create User Directory
2. Create File
3. Display Directories
4. Exit
```

```python
filename = "employees.txt"


def add_employee():

    emp_id = input("Enter Employee ID: ")
    name = input("Enter Employee Name: ")
    salary = input("Enter Salary: ")

    with open(filename, "a") as file:

        file.write(emp_id + "," + name + "," + salary + "\n")

    print("Employee record added")


def search_employee():

    emp_id = input("Enter Employee ID to search: ")

    try:

        with open(filename, "r") as file:

            for line in file:

                data = line.strip().split(",")

                if data[0] == emp_id:

                    print("\nEmployee Found")
                    print("ID:", data[0])
                    print("Name:", data[1])
                    print("Salary:", data[2])

                    return

        print("Employee not found")

    except FileNotFoundError:
```

OUTPUT

```
1. Add Employee
2. Search Employee
3. Exit
Enter choice: 1
Enter Employee ID: 1
Enter Employee Name: ATHIN
Enter Salary: 25
Employee record added

1. Add Employee
2. Search Employee
3. Exit
Enter choice: 1
Enter Employee ID: ARUN
Enter Employee Name: ARUNN
Enter Salary: 45
Employee record added

1. Add Employee
2. Search Employee
3. Exit
Enter choice: 2
Enter Employee ID to search: 1

Employee Found
ID: 1
Name: ATHIN
Salary: 25

1. Add Employee
2. Search Employee
3. Exit
Enter choice:
```

```
n = int(input("Enter number of processes: "))
m = int(input("Enter number of resources: "))

allocation = []

print("\nEnter Allocation Matrix:")

for i in range(n):

    row = list(map(int, input("P" + str(i) + ": ").split()))

    allocation.append(row)


maximum = []

print("\nEnter Maximum Matrix:")

for i in range(n):

    row = list(map(int, input("P" + str(i) + ": ").split()))

    maximum.append(row)


available = list(
    map(int, input("\nEnter Available Resources: ").split())
)


need = []

for i in range(n):

    row = []

    for j in range(m):

        row.append(maximum[i][j] - allocation[i][j])

    need.append(row)
```

OUTPUT

```
= RESTART: C:/Users/ATHIN/AppData/Local/Programs/Pytho
sssssssssssssssssssssssssssssssssssssssssssssssssssss
Enter number of processes: 3
Enter number of resources: 2

Enter Allocation Matrix:
P0: 12
P1: 2
P2: 22

Enter Maximum Matrix:
P0: 32
P1: 33
P2: 34

Enter Available Resources: 1
```

```
import threading
import time

buffer = []

size = 5

empty = threading.Semaphore(size)
full = threading.Semaphore(0)

mutex = threading.Lock()


def producer():

    for i in range(1, 6):

        empty.acquire()

        mutex.acquire()

        buffer.append(i)

        print("Produced:", i)

        mutex.release()

        full.release()

        time.sleep(1)


def consumer():

    for i in range(1, 6):

        full.acquire()

        mutex.acquire()

        item = buffer.pop(0)
```

OUTPUT

```
= RESTART: C:/Users/ATHIN/AppData/Local/Programs/Pyt
sssssssssssssssssssssssssssssssssssssssssssssssssss
Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Consumed: 3
Produced: 4
Consumed: 4
Produced: 5
Consumed: 5
Producer Consumer execution completed
```

```
import threading

counter = 0

mutex = threading.Lock()


def increment():

    global counter

    for i in range(100000):

        mutex.acquire()

        counter = counter + 1

        mutex.release()


t1 = threading.Thread(target=increment)
t2 = threading.Thread(target=increment)

t1.start()
t2.start()

t1.join()
t2.join()

print("Final Counter Value:", counter)
```

OUTPUT

```
= RESTART: C:/Users/ATHIN/AppData/Local/Programs/Pytho
sssssssssssssssssssssssssssssssssssssssssssssssssssss
Final Counter Value: 200000
```

```python
import threading
import time

read_count = 0

mutex = threading.Semaphore(1)
write_lock = threading.Semaphore(1)


def reader(reader_id):
    global read_count

    mutex.acquire()
    read_count += 1

    if read_count == 1:
        write_lock.acquire()

    mutex.release()

    print("Reader", reader_id, "is reading")
    time.sleep(1)
    print("Reader", reader_id, "finished reading")

    mutex.acquire()
    read_count -= 1

    if read_count == 0:
        write_lock.release()

    mutex.release()


def writer(writer_id):
    write_lock.acquire()

    print("Writer", writer_id, "is writing")
    time.sleep(1)
    print("Writer", writer_id, "finished writing")
```

OUTPUT

```
= RESTART: C:/Users/ATHIN/AppData/Local/Programs/Python/Python313/a
sssssssssssssssssssssssssssssssssssssssssssssssssssssssss.py
ReaderReaderReader   132   is readingis readingis reading


ReaderReaderReader   132   finished readingfinished readingfinished
reading


Writer 1 is writing
Writer 1 finished writing
Writer 2 is writing
Writer 2 finished writing
Reader-Writer execution completed
```